Ponto De Conhecimento Tecnológico

HANDS-ON #05

SQL Server — Modelação de Dados e Constraints

Base demonstrativa: SIManagerDB — Gestão de Serviços e Aplicações

Âmbito: modelação relacional, metadados, constraints e testes de integridade

Ambiente: Microsoft SQL Server 2019+ / SQL Server Management Studio

Estado: documentação consolidada a partir das oito secções aprovadas; instalação reforçada ainda pendente de teste de execução.

Índice

1. Introdução e Objetivos

2. Apresentação da SIManagerDB

3. Arquitetura e Modelo Relacional

4. Modelação e Tipos de Dados

5. Constraints e Regras de Negócio

6. Testes Práticos de Integridade

7. Validação e Conclusões

8. Anexos e Organização do Repositório GitHub

1. Introdução e Objetivos

A modelação de dados relacionais constitui uma componente fundamental na conceção e manutenção de sistemas de informação. A definição adequada de tabelas, chaves e relacionamentos permite organizar os dados, estabelecer regras de integridade e reduzir o risco de inconsistências.

Neste Hands-On #05 — SQL Server: Modelação de Dados e Constraints, da iniciativa PontoTec — Ponto de Conhecimento Tecnológico, são explorados conceitos e mecanismos de integridade de dados através de exemplos práticos em Microsoft SQL Server.

O exercício utiliza a SIManagerDB — Gestão de Serviços e Aplicações, uma base de dados demonstrativa com informação inteiramente fictícia, concebida para representar processos de gestão de aplicações, equipas técnicas, contratos, incidentes e pedidos de serviço.

1.1. Objetivos

• Analisar a organização de um modelo de dados relacional.

• Identificar chaves primárias e estrangeiras e compreender a cardinalidade dos relacionamentos.

• Explorar os tipos de constraints PRIMARY KEY, FOREIGN KEY, UNIQUE, DEFAULT e CHECK.

• Consultar os metadados do SQL Server para identificar e validar as constraints existentes.

• Executar testes controlados de integridade referencial e de regras de negócio.

• Interpretar os erros devolvidos pelo SQL Server perante operações que violam as regras definidas.

1.2. Âmbito do exercício

A SIManagerDB é constituída por 15 tabelas funcionais, relacionadas através de 21 Foreign Keys, e inclui 64 constraints dos cinco tipos analisados. O exercício contempla ainda quatro testes práticos de integridade, executados em ambiente de desenvolvimento, com tratamento de erros e reversão das operações de teste.

O objetivo não é apenas apresentar a sintaxe T-SQL, mas demonstrar como a modelação e as constraints contribuem para a consistência e fiabilidade dos dados num sistema de informação.

2. Apresentação da SIManagerDB

A SIManagerDB é uma base de dados relacional demonstrativa, desenvolvida em Microsoft SQL Server, destinada a representar um ecossistema de gestão de sistemas de informação. Utiliza dados inteiramente fictícios e constitui a base prática da série de Hands-On SQL Server da PontoTec.

2.1. Características gerais

Característica	Descrição
Sistema de gestão	Microsoft SQL Server
Compatibilidade prevista	SQL Server 2019 ou superior
Modelo	Relacional
Tabelas funcionais	15
Relacionamentos por FK	21
Constraints	64
Registos fictícios previstos	96 156
Ambiente	Desenvolvimento e demonstração técnica

2.2. Organização funcional

Estrutura organizacional e recursos técnicos

• Departamento — unidades organizacionais

• Equipa — equipas associadas aos departamentos

• Utilizador — utilizadores dos sistemas de informação

• Tecnico — técnicos associados a utilizadores e equipas

Gestão de aplicações

• Aplicacao — aplicações empresariais, criticidade e estado

• ModuloAplicacao — módulos das aplicações

• VersaoAplicacao — versões e datas de lançamento

Fornecedores, contratos e licenciamento

• Fornecedor — entidades fornecedoras

• Contrato — contratos associados a fornecedores e aplicações

• Licenca — licenças associadas aos contratos

Gestão de serviços e suporte aplicacional

• PoliticaSLA — prioridades e prazos de resolução

• Incidente — incidentes, utilizadores, técnicos e SLA

• PedidoServico — pedidos submetidos pelos utilizadores

• Intervencao — intervenções técnicas associadas a incidentes

Gestão de alterações

• Alteracao — alterações aplicacionais, tipo, estado e técnico responsável

2.3. Distribuição dos dados fictícios

Tabela	Registos previstos
Departamento	12
Equipa	10
Utilizador	600
Tecnico	120
Aplicacao	25
ModuloAplicacao	75
VersaoAplicacao	100
Fornecedor	30
Contrato	60
Licenca	120
PoliticaSLA	4
Incidente	50 000
PedidoServico	12 000
Intervencao	25 000
Alteracao	8 000
TOTAL	96 156

A geração dos dados é automatizada por instruções T-SQL, com padrões determinísticos para representar estados, prioridades, datas e associações. O total indicado corresponde ao povoamento previsto, não a uma nova contagem de execução.

2.4. Modelo relacional

O modelo utiliza chaves primárias int IDENTITY, chaves estrangeiras e constraints para definir e proteger relações. A relação Utilizador–Tecnico apresenta cardinalidade particular: cada técnico corresponde obrigatoriamente a um utilizador, mas um utilizador pode não ser técnico. A combinação de FOREIGN KEY, UNIQUE e NOT NULL em Tecnico.UtilizadorId assegura a cardinalidade 1:0..1.

2.5. Utilização no Hands-On #05

A SIManagerDB permite demonstrar estrutura e relacionamentos, definição de constraints, consulta de metadados, validação de regras de integridade e execução de testes controlados. Será reutilizada nos tutoriais seguintes da série SQL Server.

3. Arquitetura e Modelo Relacional

A SIManagerDB utiliza uma arquitetura de dados relacional em Microsoft SQL Server. Contém 15 tabelas funcionais, 15 chaves primárias, 21 relacionamentos por Foreign Key e 11 constraints UNIQUE. A integridade é complementada por CHECK, DEFAULT e regras de nulidade.

3.1. Estrutura do modelo

As chaves primárias utilizam int IDENTITY(1,1), permitindo a geração automática de identificadores. As Foreign Keys relacionam entidades e impõem integridade referencial; as regras NULL e NOT NULL determinam a obrigatoriedade da referência para o registo dependente.

3.2. Lista completa dos 21 relacionamentos

N.º	Origem	Coluna FK	Referência
1	Alteracao	AplicacaoId	Aplicacao
2	Alteracao	TecnicoId	Tecnico
3	Aplicacao	DepartamentoResponsavelId	Departamento
4	Contrato	AplicacaoId	Aplicacao
5	Contrato	FornecedorId	Fornecedor
6	Equipa	DepartamentoId	Departamento
7	Incidente	AbertoPorId	Utilizador
8	Incidente	AplicacaoId	Aplicacao
9	Incidente	SLAId	PoliticaSLA
10	Incidente	TecnicoId	Tecnico
11	Intervencao	IncidenteId	Incidente
12	Intervencao	TecnicoId	Tecnico
13	Licenca	ContratoId	Contrato
14	ModuloAplicacao	AplicacaoId	Aplicacao
15	PedidoServico	AplicacaoId	Aplicacao
16	PedidoServico	SolicitanteId	Utilizador
17	PedidoServico	TecnicoId	Tecnico
18	Tecnico	EquipaId	Equipa
19	Tecnico	UtilizadorId	Utilizador
20	Utilizador	DepartamentoId	Departamento
21	VersaoAplicacao	AplicacaoId	Aplicacao

3.3. Cardinalidades e obrigatoriedade

Predominam relações um-para-muitos (1:N). Por exemplo, um departamento pode ter várias equipas, mas cada equipa pertence obrigatoriamente a um departamento. Uma Foreign Key NULL permite ausência de referência, como Incidente.TecnicoId, quando o incidente ainda não tem técnico atribuído.

```
DepartamentoId int NOT NULL
```

```
    REFERENCES dbo.Departamento(DepartamentoId)

TecnicoId int NULL
    REFERENCES dbo.Tecnico(TecnicoId)
```

A nulidade da Foreign Key determina se a associação é obrigatória para o registo dependente; não exige que cada registo principal tenha registos dependentes.

3.4. Relacionamento Utilizador–Tecnico

```
UtilizadorId int NOT NULL
    CONSTRAINT UQ_Tecnico_Utilizador UNIQUE
    REFERENCES dbo.Utilizador(UtilizadorId)
```

Cada técnico corresponde a um utilizador existente; cada utilizador corresponde a zero ou um técnico. A Foreign Key garante existência, NOT NULL obriga a referência e UNIQUE impede que dois técnicos apontem para o mesmo utilizador.

3.5. Integridade referencial

As Foreign Keys impedem referências a registos inexistentes. O teste de inserção de Incidente com AplicacaoId = -1 foi rejeitado com erro 547, demonstrando a aplicação da regra pelo motor de base de dados.

3.6. Validação técnica dos relacionamentos

O script 01_Consultar_Relacionamentos.sql consulta nome da FK, tabelas e colunas de origem e destino, e indicadores de ativação e confiança. A execução no SSMS confirmou 21 Foreign Keys, 0 desativadas e 0 não confiáveis.

3.7. Diagrama Entidade-Relacionamento

O diagrama elaborado no SSMS representa as tabelas, chaves e relacionamentos. O artefacto previsto é SIManagerDB_DiagramaRelacional.png; a sua versão final deverá ser conferida antes da publicação. A representação gráfica não substitui a consulta aos metadados de ativação e confiança.

4. Modelação e Tipos de Dados

A escolha dos tipos de dados permite representar corretamente a informação, estabelecer restrições e assegurar compatibilidade entre colunas relacionadas.

4.1. Tipos utilizados

Tipo	Finalidade	Exemplo
int	Identificadores e inteiros	Aplicacao.AplicacaoId
tinyint	Inteiros pequenos	Tecnico.NivelSuporte
bit	Indicadores lógicos	Utilizador.Ativo
varchar(n)	Texto de comprimento variável	Incidente.Estado
nvarchar(n)	Texto Unicode	Aplicacao.Nome
decimal(p,s)	Precisão e escala fixas	Contrato.ValorAnual
date	Data sem hora	Contrato.DataInicio
datetime2(0)	Data e hora ao segundo	Incidente.DataAbertura

4.2. Identificadores e chaves primárias

```
AplicacaoId int IDENTITY(1,1)
    CONSTRAINT PK_Aplicacao PRIMARY KEY
```

IDENTITY gera valores automaticamente; PRIMARY KEY assegura unicidade e não nulidade. Os identificadores gerados não têm garantia de sequência sem lacunas.

4.3. Dados textuais

```
Nome nvarchar(120) NOT NULL
Email varchar(150) NOT NULL
```

nvarchar suporta Unicode; varchar utiliza a codificação definida pela collation e, em configurações apropriadas, pode suportar UTF-8. O comprimento declarado limita a dimensão dos valores.

4.4. Valores numéricos e precisão

```
NivelSuporte tinyint NOT NULL
    CONSTRAINT CK_Tecnico_Nivel CHECK (NivelSuporte BETWEEN 1 AND 3)

ValorAnual decimal(12,2) NOT NULL
    CONSTRAINT CK_Contrato_Valor CHECK (ValorAnual >= 0)
```

decimal(12,2) possui 12 algarismos de precisão e 2 casas decimais, permitindo até 10 algarismos na parte inteira; o máximo positivo representável é 9 999 999 999,99. A constraint impede valores negativos. É apropriado para valores monetários de precisão fixa.

4.5. Datas e informação temporal

date é utilizado em Contrato.DataInicio, Contrato.DataFim, Licenca.DataExpiracao e VersaoAplicacao.DataLancamento. datetime2(0) é utilizado em Incidente.DataAbertura, Incidente.DataResolucao, PedidoServico.DataPedido, Intervencao.DataIntervencao e Alteracao.DataAlteracao.

4.6. Nulidade e obrigatoriedade

```
TecnicoId int NULL REFERENCES dbo.Tecnico(TecnicoId)
DataResolucao datetime2(0) NULL
```

A associação do incidente a um técnico é opcional; a data de resolução pode ser nula enquanto o incidente está aberto ou em análise. A regra CK_Incidente_EstadoResolucao complementa a nulidade, relacionando estado e data.

4.7. Decisões de modelação

A integridade resulta da combinação de identificadores, chaves primárias e estrangeiras, tipos compatíveis, UNIQUE, CHECK, DEFAULT e NULL/NOT NULL. Modelação e regras de validação devem ser analisadas em conjunto.

5. Constraints e Regras de Negócio

As constraints estabelecem regras diretamente na estrutura da base de dados, independentemente das validações implementadas pelas aplicações consumidoras.

5.1. Tipos implementados

Tipo	Quantidade	Finalidade
PRIMARY KEY	15	Identificação única
FOREIGN KEY	21	Integridade referencial
UNIQUE	11	Unicidade adicional
DEFAULT	2	Valores predefinidos
CHECK	15	Regras de negócio
TOTAL	64	

5.2. PRIMARY KEY

```
AplicacaoId int IDENTITY(1,1)
    CONSTRAINT PK_Aplicacao PRIMARY KEY
```

As 15 tabelas possuem chave primária. A propriedade IDENTITY gera identificadores automaticamente, mas não assegura valores consecutivos ou sem lacunas.

5.3. FOREIGN KEY

```
AplicacaoId int NOT NULL
    REFERENCES dbo.Aplicacao(AplicacaoId)
```

Foram identificadas 21 Foreign Keys, todas ativas e confiáveis. A tentativa de inserir um incidente com AplicacaoId = -1 foi rejeitada com erro 547.

5.4. UNIQUE — 11 regras

Tabela	Coluna(s) abrangida(s)
Aplicacao	Nome
Contrato	Referencia
Departamento	Nome
Equipa	Nome
Fornecedor	Nome
Licenca	Codigo
ModuloAplicacao	AplicacaoId, Nome
PoliticaSLA	Prioridade
Tecnico	UtilizadorId
Utilizador	Email
VersaoAplicacao	AplicacaoId, NumeroVersao

```
CONSTRAINT UQ_Modulo_Aplicacao_Nome
    UNIQUE (AplicacaoId, Nome)

CONSTRAINT UQ_Versao_Aplicacao_Numero
    UNIQUE (AplicacaoId, NumeroVersao)
```

As constraints compostas impedem duplicações dentro de cada aplicação, mas permitem valores iguais entre aplicações diferentes. A unicidade de Tecnico.UtilizadorId contribui para a relação 1:0..1.

5.5. DEFAULT — 2 regras

Tabela	Coluna	Valor
Departamento	Ativo	1
Utilizador	Ativo	1

```
Ativo bit NOT NULL
    CONSTRAINT DF_Utilizador_Ativo DEFAULT (1)
```

DEFAULT aplica-se quando a coluna é omitida na inserção ou é utilizada a palavra-chave DEFAULT. Não impede outro valor válido explicitamente indicado, nem substitui NOT NULL.

5.6. CHECK — regras de negócio

As 15 constraints CHECK abrangem níveis de suporte, criticidade e estados de aplicações, datas e valores contratuais, quantidades de licenças, prazos SLA, estados e datas de incidentes, pedidos de serviço, duração de intervenções e alterações aplicacionais.

5.6.1. Valores permitidos

```
CONSTRAINT CK_Incidente_Estado CHECK (
  Estado IN ('Aberto','EmAnalise','Resolvido','Fechado')
)
```

A inserção de um incidente com Estado = Cancelado é rejeitada.

5.6.2. Datas

```
CONSTRAINT CK_Incidente_Datas CHECK (
  DataResolucao IS NULL OR DataResolucao >= DataAbertura
)
```

A data de resolução pode ser nula, mas, quando preenchida, não pode anteceder a abertura.

5.6.3. Condições entre atributos

```
CONSTRAINT CK_Incidente_EstadoResolucao CHECK (
  (Estado IN ('Aberto','EmAnalise') AND DataResolucao IS NULL)
  OR
  (Estado IN ('Resolvido','Fechado') AND DataResolucao IS NOT NULL)
)
```

Incidentes abertos ou em análise não têm data de resolução; incidentes resolvidos ou fechados devem tê-la.

5.7. Consulta e validação das constraints

O script 02_Consultar_Constraints.sql identificou 64 constraints: 15 PK, 21 FK, 11 UNIQUE, 2 DEFAULT e 15 CHECK. O resumo indicou 0 desativadas e 0 não confiáveis. Estes indicadores aplicam-se às categorias com tais propriedades nos metadados, especialmente FK e CHECK.

5.8. Importância das constraints

As constraints reduzem inconsistências, impedem referências inválidas, asseguram unicidade e validam regras entre atributos. Não substituem todas as validações funcionais: processos, autorizações e condições externas podem exigir mecanismos adicionais.

6. Testes Práticos de Integridade

Foram realizados quatro testes controlados na tabela Incidente para demonstrar a aplicação efetiva de três constraints CHECK e de uma Foreign Key.

6.1. Objetivos

• Confirmar a rejeição de valores que violam regras de negócio.

• Validar condições de consistência entre atributos.

• Demonstrar integridade referencial.

• Interpretar os erros devolvidos pelo SQL Server.

• Executar operações de teste sem pretender deixar alterações persistentes.

6.2. Metodologia

O script 03_Testes_Integridade.sql utiliza TRY...CATCH, BEGIN TRANSACTION e ROLLBACK. Cada cenário inicia uma transação, tenta inserir um registo inválido, captura o erro, reverte a transação e regista o resultado. A execução é recomendada apenas em ambiente de desenvolvimento.

6.3. Teste 01 — Data de resolução anterior à abertura

Constraint: CK_Incidente_Datas. Cenário: tentativa de inserção com DataResolucao anterior a DataAbertura. Esperado: rejeição. Obtido: erro 547. A regra impede datas cronologicamente inconsistentes.

6.4. Teste 02 — Estado inválido

Constraint: CK_Incidente_Estado. Cenário: Estado = Cancelado. Esperado: rejeição por valor fora do conjunto permitido. Obtido: erro 547.

6.5. Teste 03 — Resolvido sem data de resolução

Constraint: CK_Incidente_EstadoResolucao. Cenário: Estado = Resolvido e DataResolucao = NULL. Esperado: rejeição. Obtido: erro 547. Demonstra validação de condições entre colunas.

6.6. Teste 04 — Aplicação inexistente

Relacionamento: Incidente.AplicacaoId → Aplicacao.AplicacaoId. Cenário: AplicacaoId = -1, inexistente no ambiente de teste. Esperado: rejeição por violação de FK. Obtido: erro 547.

6.7. Resultados consolidados

Teste	Regra	Erro	Resultado
01	Datas de incidente	547	Rejeição esperada
02	Estado inválido	547	Rejeição esperada
03	Estado e data de resolução	547	Rejeição esperada
04	FK Incidente → Aplicacao	547	Rejeição esperada

Indicador	Resultado
Total de testes	4
Rejeições esperadas	4
Testes a rever	0

6.8. Tratamento de erros e reversão

TRY...CATCH permite capturar os erros de execução; ROLLBACK reverte operações abrangidas por uma transação. O estado da transação deve ser considerado, por exemplo com XACT_STATE(), para garantir uma reversão adequada quando existe transação ativa. A captura de resultados não constitui uma verificação independente de contagens após os testes.

6.9. Conclusões dos testes

As quatro rejeições esperadas confirmam a aplicação das regras selecionadas. A consulta de metadados confirma existência e configuração de constraints; os testes práticos demonstram o comportamento efetivo de regras específicas.

7. Validação e Conclusões

A validação do modelo combina a confirmação da estrutura relacional, o estado das constraints e a execução de testes práticos sobre regras selecionadas.

7.1. Validação estrutural

Elemento	Resultado
Tabelas funcionais	15
Chaves primárias	15
Foreign Keys	21
UNIQUE	11
DEFAULT	2
CHECK	15
Total de constraints	64

A tabela auxiliar sysdiagrams, quando existente, não integra a contagem funcional.

7.2. Validação dos relacionamentos

O script 01_Consultar_Relacionamentos.sql confirmou 21 Foreign Keys, 0 desativadas e 0 não confiáveis. A cardinalidade Utilizador–Tecnico foi verificada pela combinação de FOREIGN KEY, UNIQUE e NOT NULL.

7.3. Validação das constraints

O script 02_Consultar_Constraints.sql confirmou 64 constraints, 0 desativadas e 0 não confiáveis. Os indicadores de ativação e confiança devem ser interpretados de acordo com as propriedades disponibilizadas para cada categoria, especialmente FOREIGN KEY e CHECK.

7.4. Validação prática

O script 03_Testes_Integridade.sql confirmou 4 testes executados, 4 rejeições esperadas e 0 testes a rever. Nos quatro casos, o SQL Server devolveu o erro 547.

7.5. Validação da população

O script de instalação prevê 96 156 registos fictícios. O ficheiro SIManagerDB_Validacao_Comentado.sql permite consultar as contagens e explorar dados de incidentes e SLA. Nesta revisão documental, não foi apresentada uma nova evidência de contagem efetiva; o total permanece identificado como previsto.

7.6. Limitações e âmbito

• Os quatro testes práticos incidiram em três CHECK da tabela Incidente e uma FK; as demais constraints não foram individualmente testadas com operações inválidas.

• O ROLLBACK foi previsto para evitar alterações persistentes, sem comparação independente de contagens antes e depois.

• A instalação reforçada com controlo de erros foi aprovada como candidata, mas permanece pendente de teste de execução.

• Os dados são fictícios e determinísticos, não abrangendo toda a complexidade de um ambiente operacional real.

7.7. Conclusões

O exercício confirmou a existência de 15 tabelas funcionais, 21 relacionamentos e 64 constraints, bem como quatro rejeições esperadas em testes de integridade. A modelação relacional e a aplicação de constraints contribuem para a consistência, fiabilidade e sustentabilidade dos sistemas de informação. A SIManagerDB constitui uma base reutilizável para os restantes Hands-On SQL Server da PontoTec.

8. Anexos e Organização do Repositório GitHub

A disponibilização dos artefactos permite consultar o modelo, reproduzir exercícios e analisar resultados. A estrutura distingue ficheiros partilhados da base de dados dos recursos específicos de cada tutorial.

8.1. Organização proposta

```
SIManagerDB/
  README.md
  database/
    SIManagerDB_Instalacao.sql
    SIManagerDB_Validacao.sql
  hands-on-05/
    README.md
    scripts/
      01_Consultar_Relacionamentos.sql
      02_Consultar_Constraints.sql
      03_Testes_Integridade.sql
    docs/
      HandsOn05_Documentacao.pdf
      SIManagerDB_DicionarioDados_Final.csv
      SIManagerDB_DiagramaRelacional.png
  hands-on-06/
  hands-on-07/
  hands-on-08/
  hands-on-09/
```

As pastas #06–#09 representam a organização prevista e poderão ser criadas à medida que os conteúdos forem desenvolvidos.

8.2. Scripts de instalação e validação

SIManagerDB_Instalacao.sql: cria as 15 tabelas, constraints e dados fictícios. Foi preparada a versão SIManagerDB_Instalacao_Segura_SQLCMD.sql, reforçada com controlo de erros, ainda pendente de teste. SIManagerDB_Validacao.sql: consulta contagens e exemplos operacionais; a versão SIManagerDB_Validacao_Comentado.sql foi aprovada, mantendo o T-SQL original. Os nomes deverão ser uniformizados antes da publicação.

8.3. Scripts específicos

Ficheiro	Finalidade	Estado
01_Consultar_Relacionamentos.sql	Consultar 21 FKs	Validado no SSMS
02_Consultar_Constraints.sql	Consultar 64 constraints	Validado no SSMS
03_Testes_Integridade.sql	Quatro testes controlados	Validado no SSMS

8.4. Documentação técnica

HandsOn05_Documentacao.pdf: documento principal em oito secções. SIManagerDB_DicionarioDados_Final.csv: metadados de colunas, tipos, comprimentos, precisão, escala, nulidade, identidade e PK, excluindo sysdiagrams. SIManagerDB_DiagramaRelacional.png: representação gráfica do modelo, cuja versão final deverá ser conferida quanto à legibilidade antes da publicação.

8.5. Requisitos de execução

• Microsoft SQL Server 2019 ou superior e SQL Server Management Studio.

• Permissões adequadas para criação da base de dados e execução dos scripts.

• Instância de desenvolvimento ou testes, não de produção.

• Ativação de SQLCMD Mode no SSMS para a versão reforçada do script de instalação.

8.6. Sequência de utilização

Passo 1 — Preparar a base de dados com a versão de instalação validada.

Passo 2 — Verificar a população inicial com SIManagerDB_Validacao.sql.

Passo 3 — Consultar os 21 relacionamentos com 01_Consultar_Relacionamentos.sql.

Passo 4 — Consultar as 64 constraints com 02_Consultar_Constraints.sql.

Passo 5 — Executar os quatro testes com 03_Testes_Integridade.sql.

Passo 6 — Consultar o dicionário, o diagrama e a documentação técnica.

8.7. Boas práticas

Confirmar a instância antes de executar; utilizar ambientes de desenvolvimento; rever os scripts; não instalar sobre base de dados existente; verificar os resultados; executar testes de forma controlada; manter documentação e scripts sincronizados. Todos os dados são fictícios.

8.8. Continuidade da série

Hands-On	Tema previsto
#06	Joins, CTEs e Window Functions
#07	Índices e Otimização de Queries
#08	Transações e Tratamento de Erros
#09	Auditoria de Dados com Triggers e Tabelas Temporais

8.9. Considerações finais

A disponibilização conjunta dos scripts, modelo relacional, dicionário de dados e documentação favorece consulta, reprodução e evolução dos exercícios. O Hands-On #05 estabelece os fundamentos de modelação e integridade a aprofundar na série SQL Server da PontoTec.